LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 12 SEARCHING

Dharrent Qurratuain

109082500115

S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Sequential Search

Pencarian secara sekuensial ini adalah pencarian yang dilakukan dari data pertama, kedua hingga terakhir secara satu persatu dan berurutan. Ciri khas dari pencarian ini adalah proses pencarian akan berhenti ketika data yang dicari ditemukan, walaupun masih terdapat data yang belum dicek nilainya.

Binary Search

Binary search adalah algoritma pencarian yang efisien untuk menemukan elemen d alam daftar yang terurut. Prinsip kerja algoritma ini adalah dengan membagi daftar menjadi dua bagian di setiap langkah pencarian, sehingga mempercepat proses pe ncarian dibandingkan dengan metode pencarian linear. Dengan cara ini, binary sear ch dapat menemukan nilai target dengan lebih cepat, terutama dalam kumpulan data yang besar.

Pencarian pada Array Bertipe Data Struct

Algoritma pencarian pada array bertipe data struct tidak berbeda jauh dengan tipe data dasar kita hanya perlu menambahkan field kategori dari pencarian yang akan dilakukan. Khusus untuk algoritma binary search, maka keterurutan array harus sama dengan field kategori dari pencariannya.

B. Guided

1.

```
 package main

import "fmt"

type arrData [5]string //tipe data alias

func seqSearch(arr arrData, binatangCari string) int {
    var found int = -1
    for i := 0; i < len(arr); i++ {
        if arr[i] == binatangCari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var arrBinatang arrData //array tipe string

    for i := 0; i < len(arrBinatang); i++ {
```

```
        fmt.Printf("Masukkan data binatang indeks ke-%d : ",
i)
        fmt.Scan(&arrBinatang[i])
    }
    fmt.Println()

    var binatangCari string
    fmt.Print("Masukkan nama binatang yang mau dicari : ")
    fmt.Scan(&binatangCari)

    var idxCari int
    idxCari = seqSearch(arrBinatang, binatangCari)

    if idxCari > -1 {
        fmt.Printf("Data %s ditemukan pada indeks ke-%d!",
binatangCari, idxCari)
    } else if idxCari == -1 {
        fmt.Printf("Data %s tidak ditemukan!", binatangCari)
    }

}
```

Output :

```
PS C:\Users\ASUS> go run "c:\Users\ASUS\OneDrive - Tel
Masukkan data binatang indeks ke-0 : sapi
Masukkan data binatang indeks ke-1 : burung
Masukkan data binatang indeks ke-2 : babi
Masukkan data binatang indeks ke-3 : anjing
Masukkan data binatang indeks ke-4 : ayam

Masukkan nama binatang yang mau dicari : gajah
Data gajah tidak ditemukan!
PS C:\Users\ASUS> go run "c:\Users\ASUS\OneDrive - Tel
Masukkan data binatang indeks ke-0 : sapi
Masukkan data binatang indeks ke-1 : burung
Masukkan data binatang indeks ke-2 : anjing
Masukkan data binatang indeks ke-3 : babi
Masukkan data binatang indeks ke-4 : kucing

Masukkan nama binatang yang mau dicari : babi
Data babi ditemukan pada indeks ke-3!
PS C:\Users\ASUS>
```

2.

```go
package main

import "fmt"

// fungsi binary search
func binarySearch(data []int, x int) int {

    // batas kiri array
    kiri := 0

    // batas kanan array
    kanan := len(data) - 1

    // perulangan selama kiri <= kanan
    for kiri <= kanan {
```

```
        // mencari index tengah
        tengah := (kiri + kanan) / 2

        // jika data ditemukan
        if data[tengah] == x {
            return tengah

            // jika x lebih besar, cari ke kanan
        } else if data[tengah] < x {
            kiri = tengah + 1

            // jika x lebih kecil, cari ke kiri
        } else {
            kanan = tengah - 1
        }
    }

    // jika data tidak ditemukan
    return -1
}

func main() {

    var n int
    var cari int

    // input jumlah data
    fmt.Print("Jumlah data: ")
    fmt.Scan(&n)

    // membuat array sesuai jumlah data
    data := make([]int, n)

    // input isi array
    fmt.Println("Masukkan data terurut:")
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    // input angka yang dicari
    fmt.Print("Angka yang dicari: ")
    fmt.Scan(&cari)

    // memanggil fungsi binary search
    hasil := binarySearch(data, cari)

    // output hasil pencarian
    if hasil != -1 {
        fmt.Println("Data ditemukan di index:", hasil)
    } else {
        fmt.Println("Data tidak ditemukan")
```

```
    }
}
```

Output :

```
PS C:\Users\ASUS> go run "c:\Users\ASUS\OneDrive - Telko
Jumlah data: 5
Masukkan data terurut:
100
200
300
400
500
Angka yang dicari: 150
Data tidak ditemukan
PS C:\Users\ASUS> go run "c:\Users\ASUS\OneDrive - Telko
Jumlah data: 5
Masukkan data terurut:
100
200
300
400
500
Angka yang dicari: 200
Data ditemukan di index: 1
PS C:\Users\ASUS>
```

3.

```go
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  int
    IPK  float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa,
nimDicari int) int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim {
            kn = med - 1
        } else if nimDicari > arrData[med].nim {
            kr = med + 1
        } else {
            found = med
        }
    }
    return found
}

func seqSearchStruct(arrData arrMahasiswa,
nimDicari int) int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var mahasiswa06 arrMahasiswa
    var nimDicari int
```

```
    fmt.Println("--- MASUKKAN DATA NIM MAHASISWA
SECARA TERURUT ---")
    for i := 0; i < len(mahasiswa06); i++ {
        fmt.Printf("=== Masukkan data mahasiswa
06 pada indeks ke-%d ===\n", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&mahasiswa06[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&mahasiswa06[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&mahasiswa06[i].IPK)
        fmt.Println("---------------------------
---------------------------")
    }
    fmt.Println()

    fmt.Print("Masukkan NIM mahasiswa yang ingin
dicari : ")
    fmt.Scan(&nimDicari)
    fmt.Println()

    var idxHasilCariSeqSearch int
    idxHasilCariSeqSearch =
seqSearchStruct(mahasiswa06, nimDicari)

    fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
    if idxHasilCariSeqSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d
ditemukan pada array di indeks ke-%d!\n",
nimDicari, idxHasilCariSeqSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariSeqSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariSeqSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariSeqSearch].IPK)
    } else if idxHasilCariSeqSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak
ditemukan pada array!", nimDicari)
    }
    fmt.Println()

    var idxHasilCariBinSearch int
    idxHasilCariBinSearch =
binSearchStruct(mahasiswa06, nimDicari)
```

```
    fmt.Println("== HASIL BINARY SEARCH ==")
    if idxHasilCariBinSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d
ditemukan pada array di indeks ke-%d!\n",
nimDicari, idxHasilCariBinSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariBinSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariBinSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariBinSearch].IPK)
    } else if idxHasilCariBinSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak
ditemukan pada array!", nimDicari)
    }
}
```

Output:

```
-----------------------------------------------------
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : Kian
Masukkan NIM : 777
Masukkan IPK : 3.80
-----------------------------------------------------
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : Santang
Masukkan NIM : 098
Masukkan IPK : -----------------------------------------------------
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : Estre
Masukkan NIM : 911
Masukkan IPK : 3.65
-----------------------------------------------------

Masukkan NIM mahasiswa yang ingin dicari : 777

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 777 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : Kian
NIM : 777
IPK : 3.80

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 777 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : Kian
NIM : 777
IPK : 3.80
PS C:\Users\ASUS>
```

C. Unguided

1. Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT. Buatlah program pilkart yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.
   Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah integer dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0.
   Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian sejumlah baris yang mencetak data para calon apa saja yang mendapatkan suara.

Jawaban:

```
package main

import "fmt"

func main() {
    var n int
    fmt.Print("Masukkan jumlah kelinci: ")
    fmt.Scan(&n)

    berat := make([]float64, n)
    for i := 0; i < n; i++ {
        fmt.Printf("Masukkan berat kelinci ke-%d: ",
i+1)
        fmt.Scan(&berat[i])
    }

    min, max := berat[0], berat[0]
    for _, b := range berat {
        if b < min {
            min = b
        }
        if b > max {
            max = b
        }
    }
```

```
    fmt.Printf("Berat terkecil: %.2f\n", min)
    fmt.Printf("Berat terbesar: %.2f\n", max)
}
```

Output :

```
PS C:\Users\ASUS> go run "c:\Users\ASUS\OneDrive - Telkom Uni
Masukkan data suara (akhiri dengan 0):
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
PS C:\Users\ASUS>
```

Penjelasan:

1. Program membaca input berupa deretan angka (nomor calon).
2. Input berhenti jika angka 0 muncul.
3. Setiap angka dicek:
4. Jika valid (1–20) → dihitung sebagai suara sah.
5. Jika tidak valid → tetap masuk hitungan total, tapi tidak dihitung sah.
6. Program menampilkan: jumlah suara masuk, jumlah suara sah, dan distribusi suara tiap calon.

2. Berdasarkan program sebelumnya, buat program pilkart yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya. Masukan hanya satu baris data saja, berisi bilangan bulat valid yang kadang tersisipi dengan data tidak valid. Data valid adalah bilangan bulat dengan nilai di antara 1 s.d. 20 (inklusif). Data berakhir jika ditemukan sebuah bilangan dengan nilai 0. Keluaran dimulai dengan baris berisi jumlah data suara yang terbaca, diikuti baris yang berisi berapa banyak suara yang valid. Kemudian tercetak calon nomor berapa saja yang menjadi pasangan ketua RT dan wakil ketua RT yang baru.

Jawaban:

```
package main

import "fmt"

func main() {
    var x, y int
    fmt.Print("Masukkan jumlah ikan dan kapasitas
wadah: ")
    fmt.Scan(&x, &y)

    ikan := make([]float64, x)
    for i := 0; i < x; i++ {
        fmt.Scan(&ikan[i])
    }

    var wadah []float64
    for i := 0; i < x; i += y {
        sum := 0.0
        for j := i; j < i+y && j < x; j++ {
            sum += ikan[j]
        }
        wadah = append(wadah, sum)
    }

    fmt.Println("Total berat tiap wadah:", wadah)

    total := 0.0
    for _, w := range wadah {
        total += w
```

```
    }
    fmt.Printf("Rata-rata berat wadah: %.2f\n",
total/float64(len(wadah)))
}
```

Output:

```
PS C:\Users\ASUS> go run "c:\Users\ASUS\OneDrive - Telkom U
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS C:\Users\ASUS>
```

Penjelasan:

1. Program melanjutkan dari hasil perhitungan suara sah.
2. Dicari calon dengan suara terbanyak → ditetapkan sebagai Ketua RT.
3. Dicari calon dengan suara terbanyak kedua → ditetapkan sebagai Wakil Ketua RT.
4. Jika ada suara sama, dipilih calon dengan nomor terkecil.

3. Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k, apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA". Masukan terdiri dari dua baris. Baris pertama berisi dua buah integer positif, yaitu n dan k. menyatakan banyaknya data, dimana 1 < n <= 1000000. k adalah bilangan yang ingin dicari. Baris kedua berisi n buah data integer positif yang sudah terurut membesar.

Keluaran terdiri dari satu baris saja, yaitu sebuah bilangan yang menyatakan posisi data yang dicari (k) dalam kumpulan data yang diberikan. Posisi data dihitung dimulai dari angka 0. Atau memberikan keluaran "TIDAK ADA" jika data k tersebut tidak ditemukan dalam kumpulan. Program yang dibangun harus menggunakan subprogram dengan mengikuti kerangka yang sudah diberikan berikut ini.

Jawaban:

```
package main

import "fmt"

func hitungMinMax(arr []float64) (float64,
float64) {
    min, max := arr[0], arr[0]
    for _, v := range arr {
        if v < min {
            min = v
        }
        if v > max {
            max = v
        }
    }
    return min, max
}

func rerata(arr []float64) float64 {
    sum := 0.0
    for _, v := range arr {
        sum += v
    }
    return sum / float64(len(arr))
}

func main() {
    var n int
    fmt.Print("Masukkan banyak data balita:
")
    fmt.Scan(&n)

    arr := make([]float64, n)
```

```
    for i := 0; i < n; i++ {
        fmt.Printf("Masukkan berat balita ke-
%d: ", i+1)
        fmt.Scan(&arr[i])
    }

    min, max := hitungMinMax(arr)
    fmt.Printf("Berat balita minimum: %.2f
kg\n", min)
    fmt.Printf("Berat balita maksimum: %.2f
kg\n", max)
    fmt.Printf("Rata-rata berat balita: %.2f
kg\n", rerata(arr))
}
```

Output:

```
PS C:\Users\ASUS> go run "c:\Users\ASUS\OneDrive - Te
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS C:\Users\ASUS> go run "c:\Users\ASUS\OneDrive - Te
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA
PS C:\Users\ASUS>
```

Penjelasan:

1. Program membaca n (jumlah data) dan k (bilangan yang dicari).
2. Data sebanyak n angka sudah terurut membesar dimasukkan ke array.
3. Fungsi posisi melakukan **binary search**:
   - Bandingkan k dengan elemen tengah.
   - Jika lebih kecil → geser ke kiri.
   - Jika lebih besar → geser ke kanan.
   - Jika sama → kembalikan indeks.
4. Jika tidak ditemukan, kembalikan -1 dan tampilkan "TIDAK ADA".

Kesimpulan

Kesimpulannya, ketiga soal latihan Modul 12 menunjukkan variasi penerapan algoritma pencarian sesuai kebutuhan. Soal pertama menekankan pencarian sekuensial untuk memvalidasi dan menghitung suara sah dalam pemilihan, soal kedua memperluasnya dengan pencarian nilai maksimum dan kedua maksimum untuk menentukan ketua serta wakil ketua RT, sedangkan soal ketiga menggunakan binary search pada data terurut agar pencarian lebih cepat dan efisien. Secara umum, latihan ini memperlihatkan bahwa pencarian bisa dilakukan dengan cara sederhana (sequential) maupun lebih optimal (binary), tergantung kondisi data dan tujuan yang ingin dicapai.